

---

## 2. الملخص (Abstract)

يُعد الازدحام المروري (Traffic Congestion) مشكلة حرجية في البيئات الحضرية الحديثة، مما يؤدي إلى ضياع الوقت، زيادة استهلاك الوقود، وتأخر الاستجابة للحالات الطارئة. تعمل أنظمة الإشارات التقليدية (Traditional Traffic Light Systems) بناءً على فترات زمنية ثابتة (Fixed-time intervals)، بغض النظر عن الكثافة المرورية الفعلية. يقترح هذا المشروع "نظام مرور ذكي متكيف" (Smart Adaptive Traffic System) يعتمد على تقنية إنترنت الأشياء (IoT). يستخدم النظام المتحكم الدقيق (ESP32) (Microcontroller) ومستشعرات الأشعة تحت الحمراء (IR Sensors) للكشف عن الكثافة المرورية في الوقت الفعلي (Real-time)، وضبط مدة الإشارة الخضراء ديناميكياً باستخدام خوارزمية تكيفية (Adaptive Algorithm). علاوة على ذلك، يتكامل النظام مع قاعدة بيانات جوجل (Google Firebase) وتطبيق هاتف محمول مبني بـ (Flutter) للسماح بالمراقبة عن بعد وتوفير ميزة "تجاوز الطوارئ" (Emergency Override). تتيح هذه الميزة لمركبات الطوارئ فتح مسار محدد فوراً، مما يضمن مروراً آمناً وسريعاً. النتيجة النهائية هي نموذج أولي (Prototype) يعمل بكامل طاقته، يقلل من وقت الانتظار أثناء حركة المرور المنخفضة ويعطي الأولوية للحالات الطارئة بفعالية.

---

## 3. المقدمة (Introduction)

### 3.1 الخلفية والدوافع (Background and Motivation)

مع الزيادة السريعة في عدد المركبات، أصبحت أنظمة إدارة المرور التقليدية غير فعالة. غالباً ما تُبقي الإشارات ذات التوقيت الثابت السائقين في حالة انتظار عند تقاطعات فارغة بينما تكون المسارات الأخرى مزدحمة. الدافع (Motivation) وراء هذا المشروع هو استغلال قوة إنترنت الأشياء (IoT) والحوسبة السحابية (Cloud Computing) لإنشاء نظام مرور "مفكر". من خلال الانتقال من المؤقتات الثابتة (Static Timers) إلى القرارات الديناميكية المبنية على البيانات (Data-driven decisions)، يمكننا تحسين تدفق حركة المرور والتنقل الحضري بشكل كبير.

## 3.2 بيان المشكلة (Problem Statement)

تعاني أنظمة الإشارات الضوئية الحالية من مشكلتين رئيسيتين:

1. **عدم الكفاءة (Inefficiency)** الدورات الزمنية الثابتة لا تتكيف مع ظروف المرور في الوقت الفعلي، مما يسبب تأخيرات غير ضرورية.
2. **غياب أولوية الطوارئ (Lack of Emergency Prioritization)** غالباً ما تعلق سيارات الإسعاف والإطفاء في الزحام أو الإشارات الحمراء، مما قد يهدد الأرواح. لا توجد آلية تحكم عن بعد لفتح مسار لها بشكل فوري.

## 3.3 أهداف المشروع (Project Objectives)

- تصميم وتنفيذ نموذج أولي للأجهزة (Hardware Prototype) باستخدام ESP32 للتحكم في تقاطع رباعي الاتجاهات.
- تطوير خوارزمية تكيفية (Adaptive Algorithm) تضبط توقيت الإشارة بناءً على عدد المركبات الذي تكتشفه المستشعرات.
- تأسيس رابط اتصال في الوقت الفعلي (Real-time Communication) بين الأجهزة والسحابة باستخدام Firebase.
- تطوير واجهة تطبيق جوال (Mobile Application Interface) للمراقبة عن بعد والتحكم اليدوي في حالات الطوارئ.

## 3.4 النطاق والمحددات (Scope and Limitations)

**النطاق Scope** يغطي المشروع تصميم منطق التحكم المروري، واتصال إنترنت الأشياء (IoT connectivity)، وتكامل تطبيق الهاتف لتقاطع واحد رباعي الاتجاهات.

### المحددات (Limitations):

- يستخدم النموذج الأولي مستشعرات الأشعة تحت الحمراء (IR Sensors) التي قد تتأثر بضوء الشمس

المباشر (كدلالة محاكاة للحلقات الحثية أو الكاميرات في الواقع).

- يعتمد النظام على الاتصال بالإنترنت لميزات التحكم عن بعد (تطبيق الهاتف)، على الرغم من أن المنطق التكيفي المحلي (Local Adaptive Logic) يعمل دون اتصال.

## 4. نظرة عامة على النظام (System Overview)

### 4.1 الوصف عالي المستوى (High-Level Description)

يتكون النظام من ثلاث طبقات رئيسية: الطبقة الفيزيائية (Physical Layer) التي تشمل (ESP32, Sensors, LEDs)، والطبقة السحابية (Cloud Layer) المتمثلة في (Firebase Realtime Database)، وطبقة التطبيق (Application Layer) وهي تطبيق (Flutter App). تكتشف المستشعرات وجود المركبات وترسل البيانات إلى ESP32. يقوم ESP32 بمعالجة هذه البيانات لتحديد المدة المثلى للإشارة الخضراء. وفي الوقت نفسه، يزامن هذه البيانات مع Firebase. يستمع تطبيق الهاتف لتحديثات Firebase لعرض الحالة ويمكنه إرسال "أمر طوارئ" (Emergency Command) لإجبار إشارة المرور على فتح مسار محدد.

### 4.2 الوصف الوظيفي (Functional Description)

- الاستشعار (Sensing): تستخدم مستشعرات الأشعة تحت الحمراء إشارات المقاطعة (Interrupt Signals) لعد المركبات في كل مسار بدقة.
- المعالجة (Processing): يقوم ESP32 بتشغيل آلة حالات محدودة (Finite State Machine - FSM) لإدارة تسلسل إشارات المرور وتنفيذ خوارزمية التوقيت التكيفي.
- الاتصال (Communication): يستخدم النظام شبكة Wi-Fi للاتصال بقاعدة بيانات Firebase لنقل البيانات ثنائي الاتجاه (القياس عن بعد والتحكم - Telemetry & Control).
- معالجة الطوارئ (Emergency Handling): منطق مقاطعة ذو أولوية عالية (High-priority logic interrupt) يتجاوز الدورة العادية لفتح مسار لمركبات الطوارئ.

### 4.3 سيناريو حالة الاستخدام (Use-Case Scenario)

### السيناريو التشغيلي النموذجي (Typical Operating Scenario):

- الحالة الطبيعية:** عندما تقترب سيارة من المسار الشمالي، يكتشف مستشعر الأشعة تحت الحمراء ( IR Sensor) وجودها ويرسل إشارة مقاطعة إلى المتحكم الدقيق (32ESP). يقوم النظام بزيادة عداد السيارات لهذا المسار. عند انتهاء دورة الإشارة الحالية، يحسب النظام الوقت الأخضر الجديد للمسار الشمالي بناءً على عدد السيارات المنتظرة (مثلاً: 15 ثانية لـ 3 سيارات).
- حالة الطوارئ:** بينما الإشارة خضراء للمسار الشمالي، تقترب سيارة إسعاف من المسار الشرقي. يقوم المشغل (أو سائق الإسعاف) بفتح تطبيق الهاتف والضغط على زر "طوارئ الشرق". يتلقى النظام الأمر عبر السحابة (Firebase)، ويقوم فوراً بتحويل إشارة الشمال إلى اللون الأصفر (للأمان)، ثم يفتح الإشارة الخضراء للمسار الشرقي لمدة 10 ثوانٍ للسماح للإسعاف بالمرور، وبعدها يعود النظام لوضعه الطبيعي.

## 5. تصميم الأجهزة (Hardware Design)

### 5.1 قائمة المكونات (Components List)

| الكمية (Quantity) | المواصفات / الموديل<br>(Specification / Model) | اسم المكون (Component)<br>(Name) |
|-------------------|------------------------------------------------|----------------------------------|
| 1                 | ESP32 DEVKIT V1                                | متحكم دقيق (Microcontroller)     |
| 4                 | IR Obstacle Sensor Module<br>(3.3V Logic)      | مستشعرات (Sensors)               |

|        |                                  |                                  |
|--------|----------------------------------|----------------------------------|
| 12     | 5mm (Red, Yellow, Green)         | صمامات ثنائية باعثة للضوء (LEDs) |
| 12     | 220Ω (1/4 Watt)                  | مقاومات (Resistors)              |
| 1      | MB-102 Solderless Breadboard     | لوحة تجارب (Breadboard)          |
| مجموعة | Male-to-Male & Male-to-Female    | أسلاك توصيل (Jumper Wires)       |
| 1      | 5V DC via Micro-USB (Power Bank) | مصدر طاقة (Power Supply)         |

## 5.2 تبرير المكونات (Component Justification)

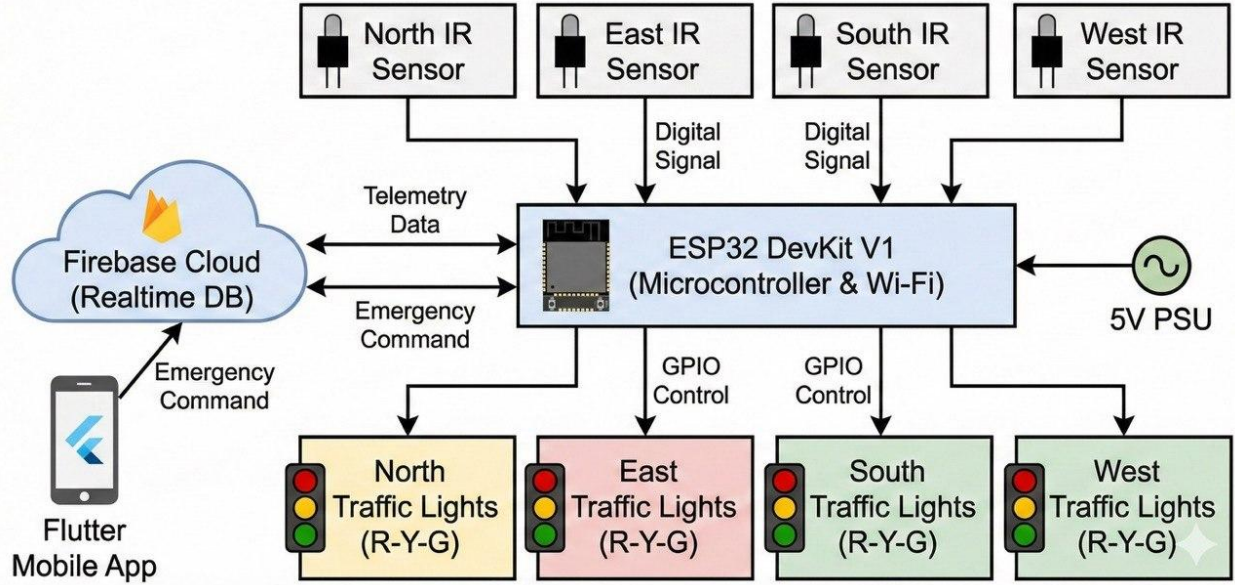
**ESP32:** تم اختياره بدلاً من **Arduino Uno** لاحتوائه على إمكانيات **Wi-Fi** و **Bluetooth** مدمجة، ومعالج ثنائي النواة (Dual-core Processor) لتعدد المهام (التعامل مع المستشعرات والمزامنة السحابية في وقت واحد)، والمزيد من دبابيس المقاطعة (Interrupt Pins).

**مستشعرات الأشعة تحت الحمراء (IR Sensors):** تم اختيارها للنموذج الأولي كبديل منخفض التكلفة لمحاكاة تطبيقات العالم الحقيقي.

**Firebase:** تم اختياره لزمّن الاستجابة المنخفض (Low Latency) في قاعدة البيانات اللحظية (Realtime Database)، وهو أمر بالغ الأهمية للاستجابة الفورية للطوارئ مقارنة بواجهات برمجة التطبيقات التقليدية (REST APIs).

### 5.3 مخطط الكتلة (Block Diagram)

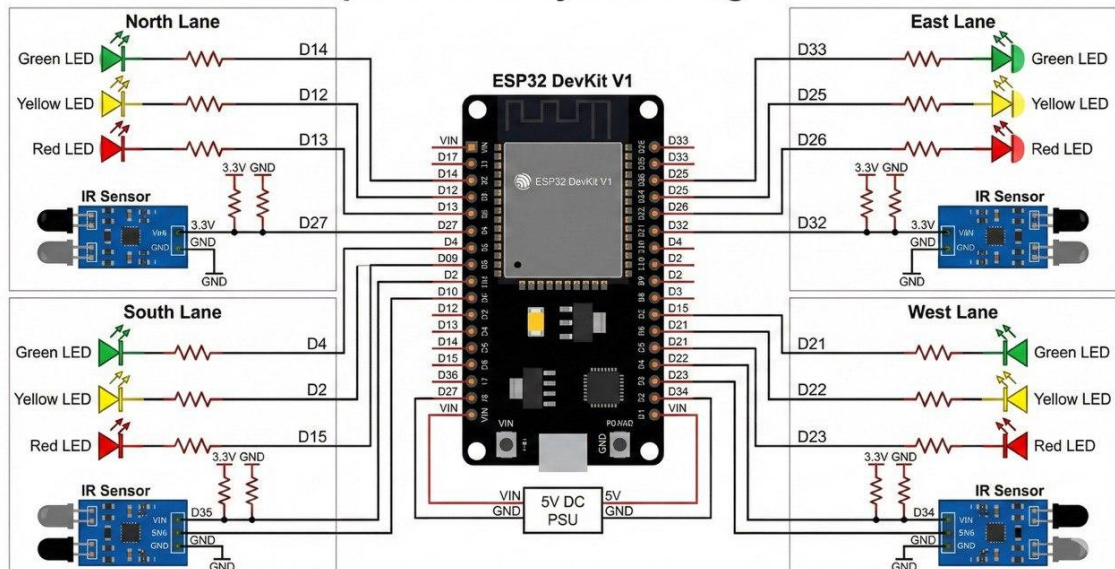
#### Smart Adaptive Traffic System using IoT & ESP32



يوضح المخطط الكتلي تدفق البيانات: تبدأ الإشارة من مستشعرات IR (مدخلات - Inputs)، تنتقل إلى وحدة المعالجة المركزية 32ESP (وحدة المعالجة - Processing Unit)، التي تتحكم في مصفوفة LEDs (مخرجات - Outputs). يتصل 32ESP لاسلكياً بسحابة Firebase لتبادل البيانات مع تطبيق الهاتف (واجهة المستخدم - UI).

### 5.4 مخطط الدائرة (Circuit Diagram)

#### Smart Adaptive Traffic System using IoT & ESP32



يوضح المخطط التفصيلي توصيل كل طرف (Pin) من أطراف 32ESP مع المكونات الخارجية، مع توضيح استخدام المقاومات لحماية الـ LEDs وتوصيل الأرضي المشترك (Common Ground).

## 6. تصميم البرمجيات (Software Design)

### 6.1 بيئة التطوير (Development Environment)

#### اللغات (Languages):

- **C++:** لبرمجة المتحكم الدقيق (Firmware).
- **Dart:** لبرمجة تطبيق الهاتف المحمول (Mobile App).

#### بيئات التطوير المتكاملة (IDEs):

- **Arduino IDE (v2.0.17):** لكتابة ورفع الكود إلى 32ESP.
- **VS Code:** لتطوير تطبيق Flutter.

#### المكتبات (Libraries):

- **32FirebaseESP:** لإدارة الاتصال بقاعدة بيانات Realtime Database.
- **WiFi.h:** لإدارة اتصال الشبكة اللاسلكية.

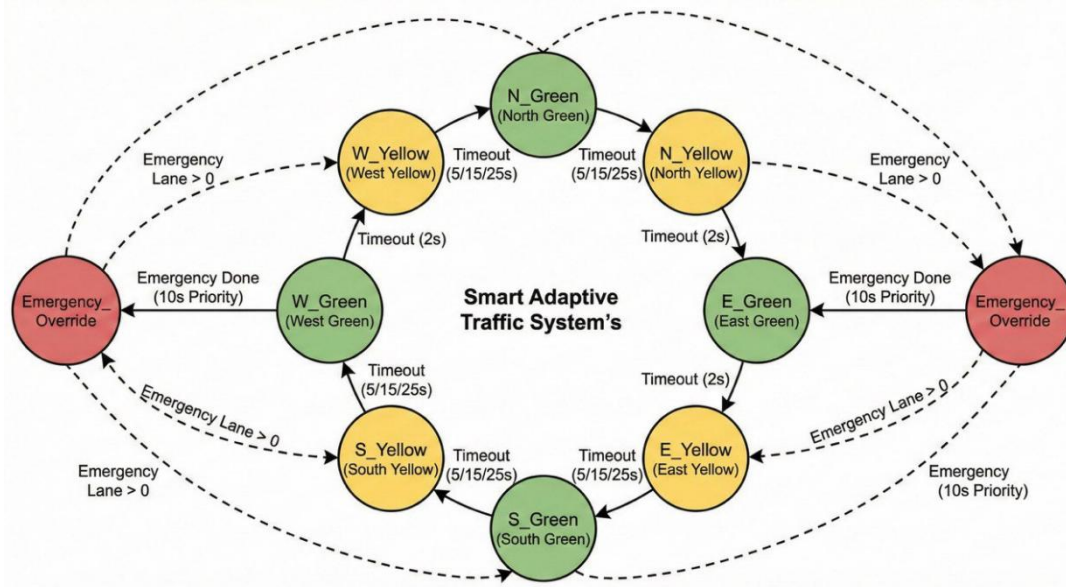
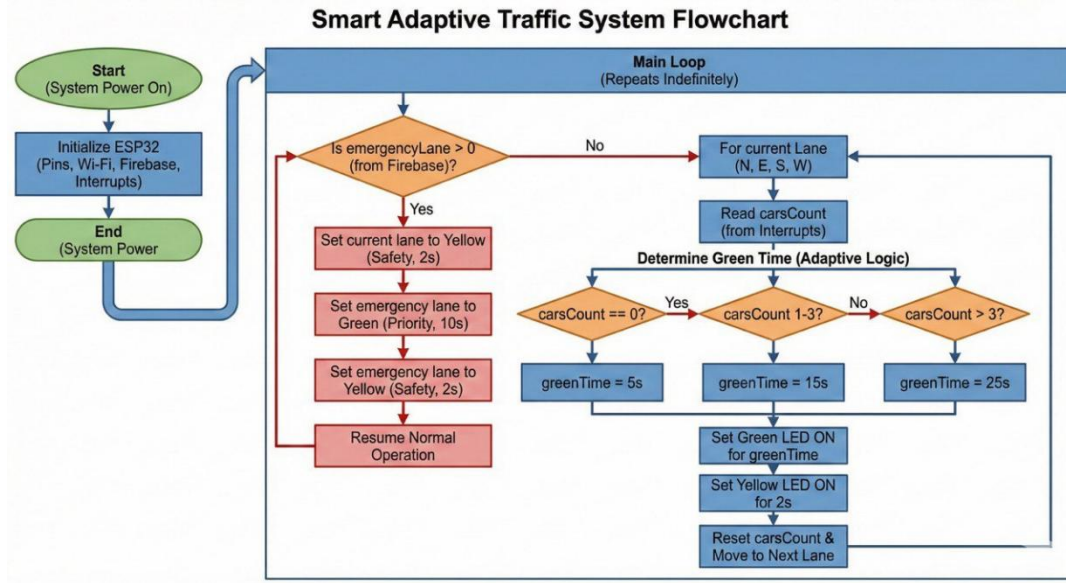
### 6.2 هندسة البرمجيات (Software Architecture)

تم تصميم البرنامج الثابت (Firmware) باستخدام بنية آلة الحالات المحدودة (Finite State Machine - FSM). يدور النظام عبر 8 حالات (أخضر/أصفر لكل من المسارات الأربعة).

- إجراءات خدمة المقاطعة (**Interrupt Service Routines - ISRs**): تُستخدم للمستشعرات لضمان عد دقيق للمركبات دون تأخير الحلقة الرئيسية (Main Loop).
- الحلقة الرئيسية (**Main Loop**): تدير انتقالات الحالة (State Transitions)، وتتحقق من إشارات الطوارئ (Emergency Flags) من السحابة، وتزامن البيانات.
- منطق منع الارتداد (**Debouncing Logic**): تم تنفيذه برمجياً (تأخير 250 مللي ثانية) لمنع العد الخاطئ الناتج عن ضوضاء المستشعر.



### 6.3 المخطط الانسيابي / مخطط الحالة (Flowchart / State Diagram)



يوضح المخطط منطق العمليات: بدءاً من التحقق من اتصال Wi-Fi، ثم الدخول في الحلقة الرئيسية (Loop) التي تفحص أولاً وجود أمر طوارئ (Emergency Flag)، وإذا لم يوجد، تنتقل لتنفيذ دورة الإشارة التكيفية بناءً على المدخلات من المستشعرات.

## 6.4 الخوارزميات (Algorithms)

### 1. خوارزمية التوقيت التكيفي (Adaptive Timing Algorithm):

بدلاً من الوقت الأخضر الثابت، يحسب النظام greenTime بناءً على متغير carsCount:

- 0 سيارات: 5 ثوانٍ (الحد الأدنى - Minimum).
- 1-3 سيارات: 15 ثانية (متوسط - Medium).
- أكثر من 3 سيارات: 25 ثانية (الحد الأقصى - Maximum).

### 2. منطق مقاطعة الطوارئ (Emergency Interrupt Logic):

عند استلام أمر الطوارئ، يحسب النظام الانتقال الأكثر أماناً. إذا كان الضوء الحالي أخضر لمسار آخر، فإنه ينتقل إلى الأصفر أولاً (بروتوكول الأمان - Safety Protocol) قبل فتح المسار الأخضر المطلوب. ثم يستمر لمدة 10 ثوانٍ قبل العودة إلى الحالة السابقة لضمان العدالة (Fairness).

---

## 10. التحديات والحلول (Challenges and Solutions)

**التحدي (Challenge):** ارتداد المستشعر (Sensor Bouncing). أنتجت مستشعرات الأشعة تحت الحمراء إشارات متعددة لمرور كائن واحد، مما تسبب في عدم دقة عد المركبات.

**الحل:** تنفيذ آلية "منع الارتداد" (Debouncing) برمجياً باستخدام دالة (milis) لتجاهل الإشارات لمدة 250 مللي ثانية بعد الكشف الأول.

**التحدي:** تأخر الشبكة (Network Latency). تسبب التأخير في اتصال الإنترنت في بطء النظام عند تحديث السحابة.

**الحل:** فصل منطق المرور عن منطق السحابة (Decoupling). تعمل إشارات المرور على مؤقت محلي (State Machine) بشكل مستقل عن اتصال Wi-Fi. تتم معالجة التحديثات السحابية بطريقة غير محظورة (Non-blocking manner) كل ثانية واحدة.

---

## 11. الخاتمة (Conclusion)

تم تصميم وتنفيذ نظام إشارات مرور ذكي بنجاح يحقق الأهداف المرجوة من حيث التكيف مع الكثافة المرورية وإدارة حالات الطوارئ. أثبت المشروع أن دمج المتحكمات الدقيقة منخفضة التكلفة (ESP32) مع الخدمات السحابية (Firebase) يمكن أن يقدم حلاً فعالاً واقتصادياً لمشاكل المرور المعقدة. تم تحقيق جميع الأهداف الوظيفية، بما في ذلك العد الدقيق للمركبات، والتحكم التكيفي في التوقيت، والتدخل الفوري للطوارئ عبر تطبيق الهاتف.

## 12. العمل المستقبلي (Future Work)

- الرؤية الحاسوبية (Computer Vision): استبدال مستشعرات الأشعة تحت الحمراء بكاميرات تعتمد على الذكاء الاصطناعي (AI) للكشف عن أنواع المركبات وليس فقط عددها.
- الطاقة الشمسية (Solar Power): تزويد النظام بالواح شمسية وبطاريات ليعمل بشكل مستقل تماماً عن الشبكة الكهربائية.
- تحليل البيانات (Data Analytics): استخدام البيانات المخزنة في Firebase لتحليل أنماط الزحام اليومية والتنبؤ بأوقات الذروة لتحسين الخوارزمية بشكل استباقي.

## 13. المراجع (References)

- [1] Espressif Systems, "ESP32 Series Datasheet," 2023. [Online]. Available: <https://www.espressif.com>.
- [2] Google, "Firebase Realtime Database Documentation," 2024. [Online]. Available: <https://firebase.google.com/docs/database>.
- [3] Arduino, "Arduino Language Reference," 2024. [Online]. Available: <https://www.arduino.cc/reference/en/>.

[4] Flutter Team, "Flutter Documentation," 2024. [Online]. Available:  
<https://flutter.dev/docs>.

---

## 14. الملاحق (Appendices)

### Appendix A: Source Code

#### Main Loop Logic (Snippet):

```
void loop() {  
    if (isEmergencyMode) {  
        // Emergency Handling Logic  
        // ... (See full code in repository)  
    } else {  
        // Normal Adaptive Cycle  
        long waitTime = (currentState % 2 != 0) ? YELLOW_TIME : currentGreenTime;  
        if (millis() - prevMillis >= waitTime) {  
            // Transition to next state  
            int nextState = (currentState + 1) % 8;  
            transitionTo(nextState);  
        }  
    }  
}
```

## Appendix B: Additional Schematics/Figures

